



IBT COLLEGE · TORONTO, CANADA

DAY 45

MODULE 3 · FRONTEND: REACT & NEXT.JS

Next.js Advanced Project

Four days of week 9 become one application today: signed in with the checks in the right places, live where it needs to be, measurably faster than it was on Monday, and findable. Then the same four topics go into your own capstone — the last checkpoint before the build weeks.



Learning Objectives

By the end of today you will be able to:



Work to a full-stack brief

Read six areas of requirement, map each to a day this week, and know how each will be demonstrated.



Decide before building

Settle who may see what, where every piece of data is fetched, and the performance budget you are aiming at.



Verify with tools

Prove protection with three attacks, live data with the network tab, and metadata from the deployed source.



Bring your capstone level

Apply this week's four topics to your own application and know exactly what remains before week 10.



Today's Agenda

1

The Project Brief

SECTION 1

Week 8 made Addis Eats work on the framework. Week 9 makes it real: signed in, live where it needs to be, fast on a phone, and findable. Today all four become one application you can hand to someone.

2

Security & Data Decisions

SECTION 2

As in week 8, decide on paper first. Three tables settle almost everything: who may see each route, where each piece of data is fetched, and what each layer of protection actually proves.

3

Build, Measure & Verify

SECTION 3

Six steps, in an order that puts security before polish. Then four kinds of verification, each of which takes minutes and none of which relies on the application looking right.

4

Your Capstone

SECTION 4

The same four topics now go into your own application. After today there is no more new framework material before the build weeks — what remains is data presentation, deployment, and finishing.

PREVIOUSLY

Day 44

Metadata & SEO

You gave every route a proper title and description, made a shared link render as a card rather than blue text, and generated a sitemap from real data. That completes week 9: live data, identity, speed and discoverability. Today they stop being four separate topics.

Key points you should be comfortable with

- Export metadata, or generate it from the data — never in a client component.
- `metadataBase` is what makes social preview images resolve at all.
- Open Graph needs a title, description, 1200 by 630 image and a URL.
- JSON-LD must match what the page actually shows.
- The rendering strategy did more for SEO than any tag.

01

SECTION 1 OF 4

The Project Brief

Week 8 made Addis Eats work on the framework. Week 9 makes it real: signed in, live where it needs to be, fast on a phone, and findable. Today all four become one application you can hand to someone.



What you are building



The same application, finished

Accounts and sessions, order history that belongs to the person viewing it, a live order status, images and fonts that do not cost four seconds, and metadata that makes a shared link worth opening.



The last checkpoint before the capstone

After today the remaining project days are your own application. This is the final assessed piece where the domain is given to you and the standard is the only variable.

FROM DAY	WHAT MUST APPEAR	PROVEN BY
41	A polled status and a debounced search	The network tab while typing
42	Sessions, three layers, scoped queries	Three attacks that all fail
43	Optimised images, fonts, measured	Lighthouse before and after
44	Metadata, previews, sitemap, JSON-LD	The deployed page source

The requirements

AREA	REQUIREMENT
Accounts	Sign in, sign out, a session in a flagged cookie
Protection	Middleware, page check, and a check in every action
Ownership	Every query scoped to the session, never to a sent id
Live data	One polled query and one debounced search
Performance	A measured improvement, documented
Discovery	Metadata, previews, sitemap and structured data



Six areas, all verifiable

Not one of these is a matter of opinion. Each can be demonstrated in front of the room — which is exactly how they will be assessed.

🔧 Everything before still applies Routing, layouts, rendering strategy and the server-client boundary from week 8 are assumed. Breaking them to satisfy something new is not a trade you are allowed.

What "done" means here

NOT DONE	DONE
Signed-in pages look right	Three attacks all fail
"It feels faster"	A recorded before and after
A spinner on every arrival	Server data seeds the client query
A title on the home page	A distinct description on every route
The link shares as blue text	A card with an image and a price
Works on your laptop	Works on a deployed preview



Evidence, not assertion

Each line names a thing you can show. If a claim in your README has no screenshot, number or command behind it, treat it as not yet done.

⚠ Deploy before you present Social previews, absolute URLs and environment variables only behave correctly on a real deployment. A localhost demonstration hides exactly the problems this project is about.

How it is marked

CRITERION	WEIGHT	BAND
Authentication and authorisation	30%	A–F
Data fetching, server and client	20%	A–F
Measured performance improvement	20%	A–F
Metadata, previews and discovery	15%	A–F
Documentation and walkthrough	15%	A–F



Security is the largest slice

A beautiful, fast application that lets one customer read another's orders fails this project. Get the checks right first and optimise afterwards.

 **The documents count** AUTH.md, DATA.md and PERF.md from this week are deliverables, and fifteen per cent of the mark is for them and the walkthrough together.

02

SECTION 2 OF 4

Security & Data Decisions

As in week 8, decide on paper first. Three tables settle almost everything: who may see each route, where each piece of data is fetched, and what each layer of protection actually proves.

Decision one: who may see what

ROUTE	WHO	PROTECTED BY	WHAT THAT PROVES
/menu	Everyone	Nothing	It is public, and should stay indexable
/orders	The owner	Middleware + page + scoped query	Signed in, and only their own records
/checkout	Any signed-in	Middleware + page	Signed in — the write checks the rest
/kitchen	Staff only	Page + role check in every action	Signed in, and permitted

✎ **Write the fourth column** The habit that matters is the last column. "Protected by middleware" is not an answer — middleware only proves a cookie exists. Saying exactly what each layer establishes is what shows you where the gap is, and the gap is almost always an action that trusts an id it was handed.

Decision two: where data is fetched

DATA	WHERE, AND WHY
Menu and dishes	Server — public, indexable, cacheable
Order history	Server — private, scoped to the session
Order status	Server first, then polled on the client
Menu search	Client — triggered by typing, debounced
Cart	Client store — it never leaves the browser



Server unless the person triggers it

That single rule decides four of these five rows. The exceptions are the ones a person types into or that must keep refreshing on their own.

Then seed the exception Where a client query is unavoidable, render the first result on the server and pass it in as fallbackData. Nobody should watch a spinner for data you already had.

Decision three: the performance budget

MEASURE	TARGET	YOURS
LCP, throttled	Under 2.5s	—
CLS	Under 0.1	—
First Load JS, /menu	Under 120 kB	—
Largest image	Under 150 kB	—
Lighthouse performance	90 or better	—



Fill the third column twice

Once before you optimise and once after. A budget with only the final number in it proves nothing — the improvement is the evidence, not the score.

⚠ Miss a target honestly If a number will not come down, write why. A documented failure you understand is worth more than a number you cannot explain.

The three documents



AUTH.md

Every route and action, its layers, and what each layer proves. Plus the three attacks and what stopped each one.



DATA.md

Every query: server or client, its key, its refresh rule, and one line of reasoning for each.



PERF.md

The budget table with both columns filled, and what caused each change between them.

```
one line per row is enough

/orders  middleware -> cookie exists
         page       -> session verified
         query      -> getOrdersFor(session.id)
         action     -> ownership re-checked
```

 **Written while you build** These are notes made as you go, not documentation written on Friday. Filling a row is how you notice the layer you have not added yet.

03

SECTION 3 OF 4

Build, Measure & Verify

Six steps, in an order that puts security before polish. Then four kinds of verification, each of which takes minutes and none of which relies on the application looking right.

The order of work

#	BUILD IN THIS ORDER
1	Sessions, sign-in and sign-out working end to end
2	The three layers on every private route and action
3	Scoped queries, then run the three attacks
4	The polled status and the debounced search, both seeded
5	Measure, then optimise images, fonts and scripts
6	Metadata, previews, sitemap — then deploy and verify



Security first, deliberately

Optimising a page you later restructure for authorisation wastes the optimisation. Get the checks in place while the code is still easy to move.

🔧 **Measure at step five** Take the baseline after the features are complete and before you optimise anything. A baseline taken too early measures a different application.

Verifying the protection

the three attacks, from the browser console

```
// 1 – signed out
await cancelOrder("ord_812");
// expect: refused, not a crash

// 2 – signed in as someone else
await cancelOrder("<another account's order id>");
// expect: refused on ownership, not on session

// 3 – a crafted sign-in link
/signin?next=https://example.com
// expect: lands on /, not on example.com
```



Name the line that stopped it

For each attack, write down which line refused it. If you cannot point at one, the request was refused by luck or by the interface — neither counts.

🔪 **Also try the URL** Change the id in /orders to another account's. A scoped query makes this impossible; an unscoped one makes it trivial.

Verifying the rest

AREA	HOW TO CHECK	WHAT YOU WANT TO SEE
Live data	Network tab while typing	One request per pause, not per keystroke
Seeding	Load the order page	Data on arrival, no spinner at all
Performance	Lighthouse on the deployed build	Both numbers recorded, LCP down
Metadata	View source on the deployed page	Absolute og:image URLs, distinct descriptions
Discovery	Open /sitemap.xml and /robots.txt	Real dishes listed, private routes absent

🔪 **None of this is looking at the application** Every check above uses a tool rather than an opinion: the network tab, a Lighthouse run, the page source, two generated files. That is deliberate — an application can look completely finished and fail all five.

Presenting it

MINUTES	WHAT TO COVER
0–1	The deployed application, signed in, one order placed
1–3	Your AUTH.md table, and the three attacks failing live
3–5	The network tab while searching, and the seeded order page
5–7	PERF.md — the before and after, and what caused it
7–8	A shared link previewing properly, and questions

→ Run the attacks live

Nothing demonstrates that authorisation works like calling the action in front of the room and watching it refuse. It takes twenty seconds and it is the most convincing thing you can do.

🔪 **Present the deployment** From the preview URL, not localhost. It is what you were assessed on and it is the only place the previews and environment variables are real.

04

SECTION 4 OF 4

Your Capstone

The same four topics now go into your own application. After today there is no more new framework material before the build weeks — what remains is data presentation, deployment, and finishing.

Applying week 9 to your own project

FROM DAY	WHAT YOUR CAPSTONE NEEDS
41	Any screen that polls or searches, done with a library
42	Sign-in, and the three layers on every private route
43	A measured pass over images, fonts and scripts
44	Titles, descriptions, previews and a sitemap
All	AUTH.md, DATA.md and PERF.md for your own routes



Not every project needs all of it

If nothing in your application polls, do not invent something that does. Write down that decision instead — knowing which tools you do not need is part of the assessment.

⚠ Authentication is not optional Almost every capstone has something private. If yours genuinely has nothing behind a sign-in, discuss it with the instructor today rather than on Day 49.

Where your capstone should be

BY THE END OF TODAY	STATUS
Every route reachable on Next.js	Since Day 40
Layouts and rendering strategy chosen	Since Day 40
Server and client boundary documented	Since Day 40
Sign-in and protected routes	This week
One measured optimisation pass	This week
Metadata on every route	This week



Behind? Say so now

Week 10 assumes all six rows are done. A capstone still missing routes on Day 46 will not be finished by Day 49 without help — and help is available today.

Thin screens are fine A route that exists and renders a heading is progress. An empty route map on Day 46 is not.



What week 10 adds

DAY	TOPIC	WHAT YOUR CAPSTONE GAINS
46	Data representation	Charts, tables, pagination, and honest empty states
47	Maps, real-time & formats	Leaflet, live updates, ETB and date formatting
48	Events & deployment	Interaction detail, and a live URL on Vercel
49	Capstone build	The final day of building, with support in the room
50	Review & assessment	Presentation, code review, module assessment

🔪 **Five days left** Three of them introduce new material and apply it, one is a build day, and the last is assessment. That is less time than it sounds — which is why anything still unfinished from weeks 8 and 9 should be closed this weekend rather than carried forward.

The standard to hold



Secure by structure

Queries scoped to the session and checks inside the write, so the whole class of bug is impossible rather than merely absent.



Fast by measurement

A number before, a number after, and a sentence naming what moved it. Never "it feels quicker".



Findable by default

Server-rendered, described honestly, and worth opening when the link appears in a group chat.

🔍 **The question behind all three** What happens if someone uses this without the interface — on a slow phone, from a search result, or with a request they wrote themselves? An application that answers all three well is one you can put your name to.

⚠️ **Tomorrow** Day 46 opens week 10 with data representation: charts, tables and data grids, pagination and infinite scroll, and the skeleton, empty and error states that make data feel trustworthy.

05

WRAP-UP

Summary & Next Steps

Six areas of requirement, three decision tables, four kinds of verification, and three short documents. Week 10 is data, deployment and the capstone.

Day in Review



The brief

Addis Eats with accounts, live data, measured performance and full metadata — the last assessed project with a given domain.



Three decisions

Who may see each route and what each layer proves, where every query runs, and the performance budget you will measure against.



Security is 30%

A fast, beautiful application that leaks one customer's orders to another fails. Build the checks before the polish.



Verify with tools

Three attacks, the network tab, a Lighthouse run and the deployed page source. None of it is looking at the interface.



Three documents

AUTH.md, DATA.md and PERF.md, written while you build. Filling a row is how you find the layer you forgot.



The capstone

Six rows should be true of it by tonight. Week 10 assumes all of them and there are only five days left.

Review Questions

ANSWER THESE BEFORE YOU LEAVE — DISCUSS IN PAIRS

- 1 For one private route in your build, what does each of the three layers actually prove?
- 2 Which query in your application runs on the client, and why can it not run on the server?
- 3 What were your LCP numbers before and after, and which change did most of it?
- 4 Show the line that refuses an action called with another account's id.
- 5 Why must the performance baseline be taken after the features are finished?
- 6 What is still missing from your capstone, and when will it be done?

In-Class Exercise



Build the week 9 project and level your capstone

DELIVERABLE

Addis Eats deployed with accounts, three-layer protection, a polled status and debounced search, a measured optimisation pass and full metadata — with AUTH.md, DATA.md and PERF.md. Plus your capstone brought to the same six-row standard.

Steps

- Write the three decision tables before touching any code.
- Build steps one to three, then run the three attacks and record what stopped each.
- Add the polled status and debounced search, both seeded with server data.
- Take the performance baseline, optimise, and fill both columns of the budget.
- Add metadata, previews, sitemap and robots, then deploy and verify from the source.
- Apply the same four topics to your capstone and write its three documents.

PROVIDED FILES

AUTH.md Route, layers, and what each layer proves. Plus the attacks.

DATA.md Each query: server or client, key, refresh rule, reasoning.

PERF.md The budget table, both columns filled, with causes.

README.md The deployed URL, and how to run it from a fresh clone.

Tip Deploy on the first morning, not the last afternoon. Absolute URLs, environment variables and social previews all behave differently in production, and finding that out early costs minutes rather than an evening.



Homework & Reading



Read the reading sheet

The full brief, the rubric, the three decision tables, the verification commands and the capstone checklist.



Finish the project

Complete all six areas, deploy it, and push with the three documents and a working URL.



Level your capstone

Get all six rows true of your own application before Monday.



Read ahead

Skim the Recharts quick start — week 10 opens with charts, tables and data presentation.

 **Reading sheet** Today's revision, full reading list and the mini-project are in `CodeOps_M3_D45_Reading_Sheet_NextJS_Advanced_Project.docx`



Learning Resources

DOCS

Production checklist

nextjs.org

Everything to verify before calling a build finished — useful as a rubric.

DOCS

Authentication

nextjs.org

The framework's guidance on layering checks, worth rereading today.

GUIDE

Deploying to Vercel

vercel.com

Preview deployments and environment variables, needed before you present.

READ

OWASP top ten

OWASP

Broken access control is number one — exactly what the three attacks test.

DOCS

Lighthouse scoring

Chrome for Developers

What each metric weighs, so your budget targets are informed.

VIDEO

Shipping a Next.js app

Vercel

From local build to deployed URL with environment configuration.